

TCP (TRANSMISSION CONTROL PROTOCOL)

Nama : Syahril Arfian Almazril

Nim : 103032300013

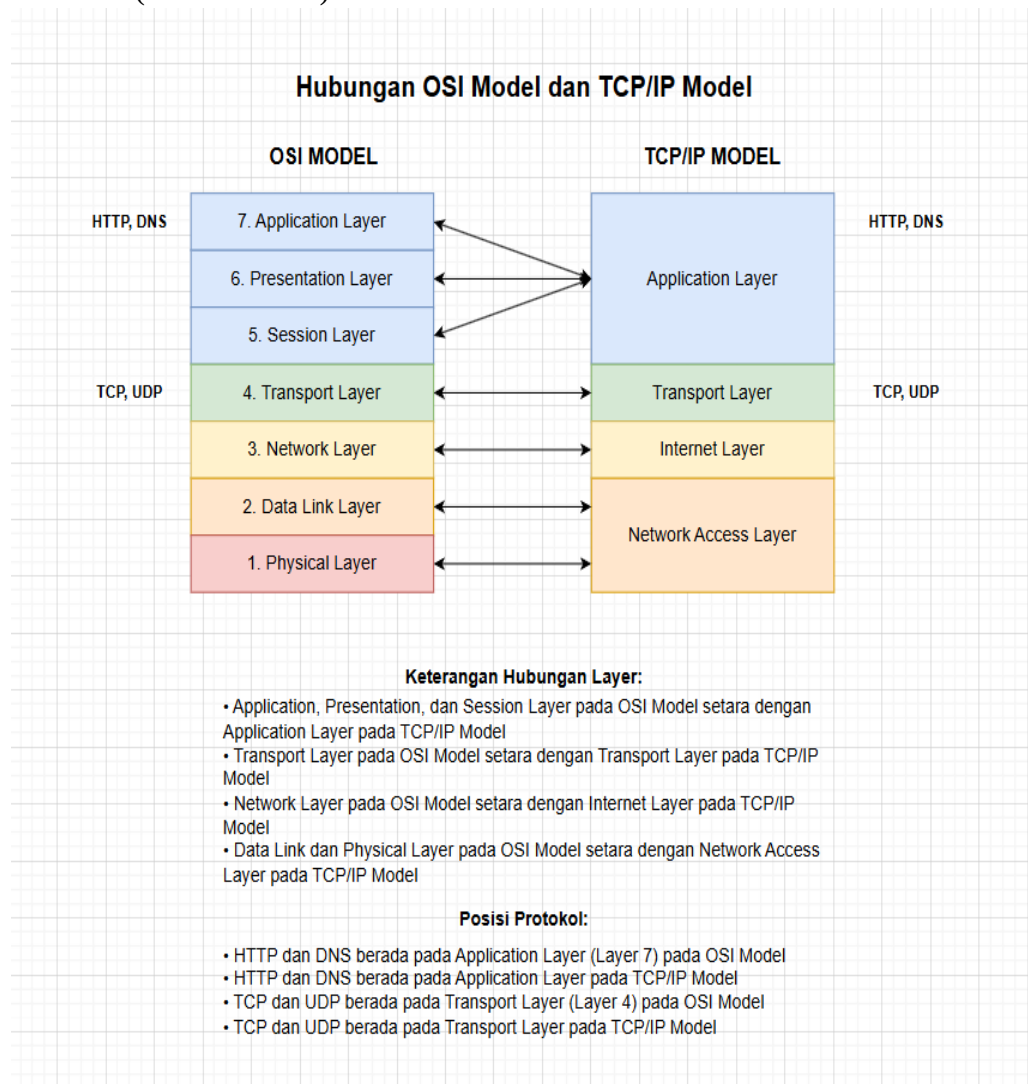
Kelas : IT-47-04

Kode Asprak : STO

A. Teori

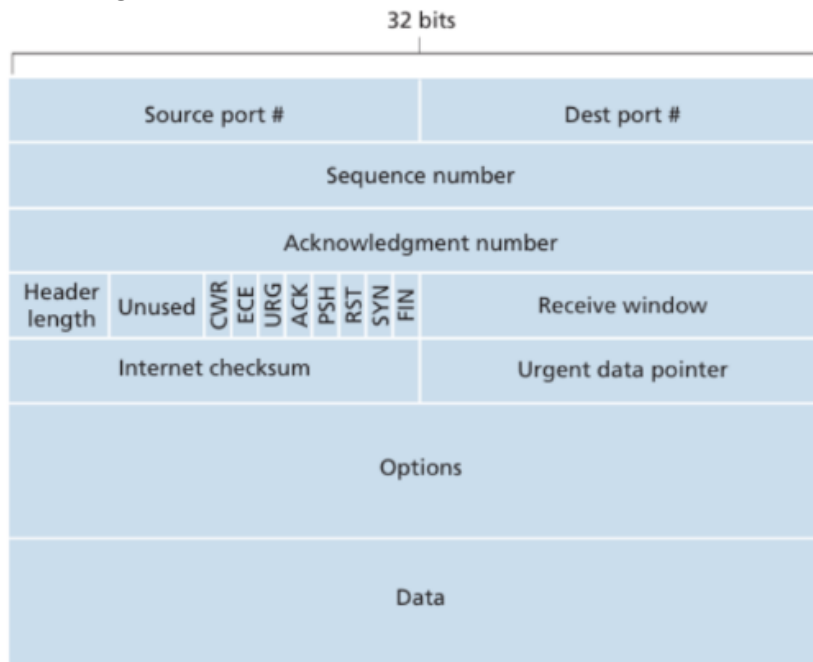
1. Saat ini anda seharusnya sudah mengetahui apa itu OSI Model dan TCP/IP Model. Tuliskan dan jelaskan hubungan layer pada OSI Model dengan layer pada TCP/IP Model (contoh Transport Layer pada OSI Model setara dengan Transport Layer pada TCP/IP Layer) buatkan secara visual. Serta sebutkan protokol HTTP, DNS, UDP dan TCP berada pada layer mana untuk setiap model

Jawaban : (buat di draw.io)



TCP (TRANSMISSION CONTROL PROTOCOL)

2. Perhatikan gambar dibawah ini



Berdasarkan gambar tersebut jawablah pertanyaan dibawah ini:

- a. Apa yang dimaksud dari gambar diatas?

Gambar tersebut adalah struktur header dari segmen TCP (Transmission Control Protocol). Header ini digunakan dalam komunikasi jaringan untuk memastikan pengiriman data antar device itu bisa berlangsung dengan andal dan berurutan. Struktur ini berisi berbagai informasi penting seperti nomor port, nomor urutan, flag kontrol, dan lainnya.
- b. Jelaskan fungsi dari Source port dan Destination port dalam gambar tersebut!

sebutkan port-port yang digunakan untuk protocol HTTP, HTTPS, UDP dan TCP!

 - Source Port: Port asal pengirim — digunakan untuk mengidentifikasi aplikasi yang mengirim data dari sisi pengirim.
 - Destination Port: Port tujuan — digunakan untuk mengidentifikasi aplikasi yang menerima data di sisi penerima.

Contoh Port yang Digunakan:

- HTTP menggunakan port 80. Port ini adalah biasanya default untuk komunikasi web tidak terenkripsi.
- HTTPS menggunakan port 443. Port ini digunakan untuk komunikasi web yang aman dan terenkripsi menggunakan SSL/TLS.
- UDP merupakan protokol transport yang port-nya bervariasi tergantung pada aplikasi. Contohnya, DNS menggunakan port 53 saat memakai UDP.
- TCP adalah protokol transport. Banyak protokol yang menggunakan TCP sebagai dasar pengirimannya, seperti HTTP, HTTPS, FTP dan lainnya.

- c. Apa yang dimaksud dengan sequence number dan acknowledgment number?

Bagaimana keduanya digunakan dalam transfer data? Apa yang terjadi jika sequence number bernilai 1000 dan acknowledgement number bernilai 2000?

TCP (TRANSMISSION CONTROL PROTOCOL)

- Sequence Number: Menunjukkan nomor urutan byte pertama dari segmen data yang dikirim. Digunakan untuk menjaga agar data diterima berurutan.
- Acknowledgment Number: Menunjukkan byte selanjutnya yang diharapkan oleh penerima. Digunakan untuk mengonfirmasi penerimaan data.

Contoh:

- Sequence Number = 1000 artinya byte pertama dari data yang dikirim dimulai dari 1000.
- Acknowledgment Number = 2000 artinya penerima mengonfirmasi telah menerima hingga byte 1999 dan menunggu byte 2000 selanjutnya.

d. Jelaskan peran flags (SYN, ACK, FIN, RST, PSH, URG, ECE, CWR) pada gambar tersebut! Berikan contoh penggunaan dari flags tersebut!

- SYN: Digunakan untuk memulai koneksi. Contoh penggunaannya adalah saat proses 3-way handshake (SYN → SYN-ACK → ACK) untuk membangun koneksi antara dua perangkat.
- ACK: Berfungsi untuk mengonfirmasi penerimaan data. Biasanya digunakan dalam hampir semua segmen setelah koneksi terbentuk untuk memastikan bahwa data telah diterima dengan benar.
- FIN: Digunakan untuk mengakhiri koneksi. Contohnya ketika salah satu pihak ingin menutup koneksi TCP secara teratur.
- RST: Berfungsi untuk mereset koneksi. ini dikirimkan jika terjadi error atau koneksi yang tidak valid, misalnya port tidak terbuka.
- PSH: Digunakan untuk mendorong data ke aplikasi. Misalnya, saat data perlu langsung diteruskan ke aplikasi penerima tanpa menunggu buffer.
- URG: jika adanya data penting (urgent) yang harus segera diproses. Flag ini biasanya hanya untuk situasi kontrol darurat.
- ECE: Menunjukkan bahwa telah terjadi kemacetan jaringan. Digunakan dalam mekanisme congestion control untuk mengatur aliran data.
- CWR: Merupakan respon terhadap flag ECE, menandakan bahwa pengirim telah mengurangi ukuran jendela (window) sebagai tanggapan terhadap kemacetan yang terdeteksi.

e. Apa fungsi Window Size dari gambar tersebut? Serta jelaskan perbedaan antara checksum dan urgent pointer dalam gambar tersebut!

- Window Size: Menentukan jumlah byte yang dapat diterima oleh penerima tanpa mengirim acknowledgment kembali. Digunakan dalam flow control.
- Semakin besar nilai ini, semakin banyak data yang bisa dikirim sekaligus.

Perbedaan Checksum dan Urgent Pointer:

- Checksum: Untuk mendeteksi error dalam header dan data selama transmisi.
- Urgent Pointer: Menunjukkan posisi akhir dari data urgent dalam segmen jika URG flag aktif.

3. Jelaskan apa yang dimaksud dengan:

a. TCP Flow Control dan TCP Congestion Control

- Flow control adalah mekanisme untuk menghindari pengiriman data yang berlebihan ke penerima. Tujuannya agar pengirim tidak membanjiri buffer penerima. Mekanisme ini menggunakan Window Size (ukuran jendela) yang

TCP (TRANSMISSION CONTROL PROTOCOL)

dikirim oleh penerima, yang menunjukkan berapa banyak data yang dapat diterima sebelum harus mengirim acknowledgment.

- Congestion control bertujuan untuk menghindari kemacetan (congestion) di jaringan. TCP mengontrol laju pengiriman data berdasarkan kondisi jaringan. Proses ini melibatkan algoritma seperti Slow Start, Fast Retransmit dll nya.

b. RTT, sampleRTT, estimatedRTT dan devRTT

- RTT (Round-Trip Time)

Waktu yang dibutuhkan oleh sebuah segmen untuk pergi dari pengirim ke penerima dan kembali lagi (melalui acknowledgment).

- sampleRTT

Merupakan pengukuran aktual RTT untuk satu segmen. Diukur secara langsung selama komunikasi.

- estimatedRTT

Merupakan rata-rata bergerak dari sampleRTT, digunakan untuk memperkirakan RTT saat ini dan menghindari fluktuasi ekstrem.

- devRTT (Deviation RTT)

Mengukur variasi dari sampleRTT terhadap estimatedRTT. Digunakan untuk menentukan tingkat ketidakpastian.

c. TimeoutInterval

TimeoutInterval adalah waktu maksimum yang ditunggu pengirim untuk menerima acknowledgment sebelum mengirim ulang segmen.

Rumus penentuan TimeoutInterval:

$$\text{TimeoutInterval} = \text{estimatedRTT} + 4 * \text{devRTT}$$

Timeout ini sangat penting agar retransmisi tidak terjadi terlalu cepat atau terlalu lambat.

d. TCP Throughput dan avg TCP Throughput

- TCP Throughput

Adalah jumlah data yang berhasil dikirim per satuan waktu dalam koneksi TCP. Dipengaruhi oleh kapasitas jaringan, congestion, dan loss.

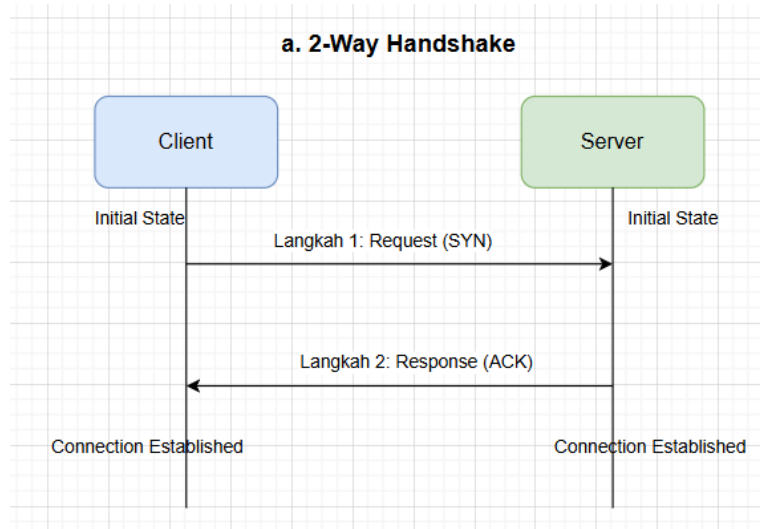
- avg TCP Throughput (average TCP Throughput)

Merupakan rata-rata kecepatan pengiriman data TCP dalam periode tertentu. Ini menunjukkan efisiensi TCP dalam menggunakan kapasitas jaringan.

4. Buatlah ilustrasi dan jelaskan perbedaan antara:

TCP (TRANSMISSION CONTROL PROTOCOL)

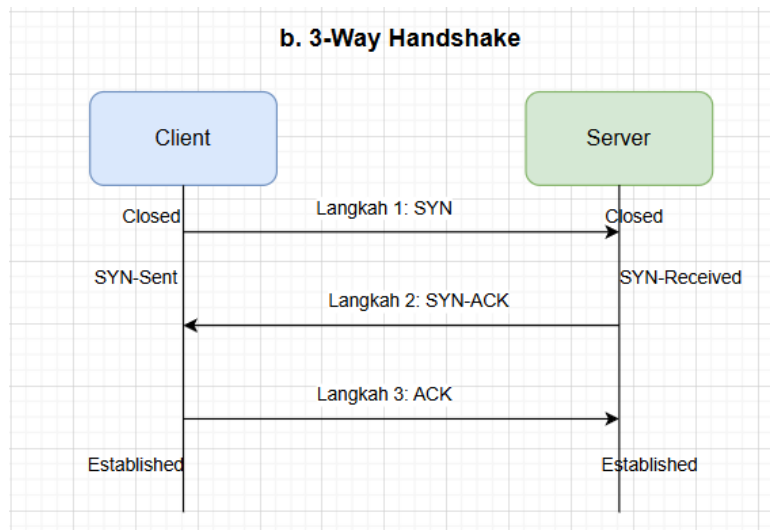
a. 2-way handshake scenarios



Penjelasan:

- Hanya terdiri dari dua pesan: Request (SYN) dan Response (ACK)
- Client mengirimkan permintaan koneksi (SYN)
- Server langsung menerima koneksi dan mengirim acknowledgment (ACK)
- Koneksi terbentuk setelah 2 langkah
- Lebih sederhana dan cepat, tetapi kurang handal

b. 3-way handshake scenarios



Penjelasan:

- Terdiri dari tiga pesan: SYN, SYN-ACK, dan ACK
- Client mengirimkan permintaan sinkronisasi (SYN) dengan sequence number awal
- Client mengirim ACK final untuk mengonfirmasi penerimaan SYN dari server
- Koneksi terbentuk setelah 3 langkah
- Lebih handal, menyediakan verifikasi dua arah

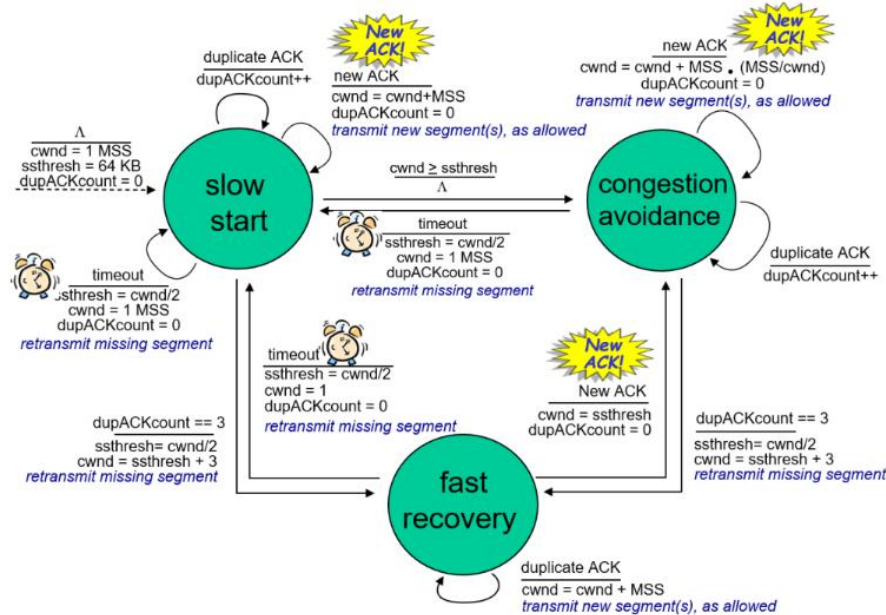
Perbedaan Utama:

- Jumlah langkah: 2-Way menggunakan 2 langkah, 3-Way menggunakan 3 langkah

TCP (TRANSMISSION CONTROL PROTOCOL)

- Keandalan: 3-Way lebih handal karena konfirmasi dua arah
- Keamanan: 3-Way lebih aman terhadap serangan dan koneksi palsu
- Kecepatan: 2-Way lebih cepat karena memerlukan lebih sedikit pertukaran pesan
- Penggunaan: TCP menggunakan 3-Way, protokol lebih sederhana (2 way)
- Status koneksi: 3-Way memiliki state machine yang lebih kompleks.

5. Perhatikan gambar berikut yang merupakan berkaitan TCP Congestion Control



Berdasarkan alur pada gambar di atas, jawablah pertanyaan dibawah ini

a. Jelaskan fase slow start dalam mekanisme congestion control TCP seperti yang ditunjukkan pada gambar! Apa tujuan dari fase ini dan bagaimana TCP mengatur laju pengiriman data selama fase ini?

Fase Slow Start adalah fase awal saat koneksi TCP baru dibentuk. Tujuannya adalah untuk membangun kecepatan pengiriman data secara bertahap agar tidak langsung membanjiri jaringan.

- Inisialisasi:
 - o $cwnd = 1 \text{ MSS}$ (Maximum Segment Size)
 - o $ssthresh$ diset (misalnya 64 KB)
 - o $dupACKcount = 0$
- Pertambahan cwnd:
 - o Setiap ACK baru diterima $\rightarrow$ cwnd bertambah secara eksponensial ($cwnd = cwnd + MSS$)
- Tujuan:
 - o Mendeteksi kapasitas jaringan secara hati-hati
 - o Menghindari kemacetan sejak awal
- Jika terjadi timeout:
 - o Dianggap ada paket hilang $\rightarrow$ kembali ke slow start dengan $cwnd = 1 \text{ MSS}$, dan $ssthresh = cwnd / 2$

TCP (TRANSMISSION CONTROL PROTOCOL)

- b. Bagaimana TCP beralih dari fase slow start ke fase congestion avoidance?
Apa kondisi yang menyebabkan transisi ini, dan bagaimana perilaku TCP berubah setelah transisi tersebut?

- Kondisi transisi:

- o Ketika nilai $cwnd \geq ssthresh$
- o TCP berpindah dari pertumbuhan eksponensial ke pertumbuhan linier

- Perilaku setelah transisi (fase Congestion Avoidance):

- o $cwnd$ bertambah lebih lambat: $cwnd = cwnd + MSS * (MSS / cwnd)$
- o TCP mencoba menjaga keseimbangan antara efisiensi dan menghindari kemacetan

- c. Jelaskan mekanisme fast recovery dalam TCP seperti yang ditunjukkan pada gambar! Apa tujuan tujuan dari fast recovery dan bagaimana TCP merespons packet loss selama fase ini?

Fast Recovery adalah mekanisme yang digunakan ketika TCP mendeteksi packet loss melalui 3 duplicate ACK (bukan timeout).

- Kondisi masuk ke fast recovery:
 - o $dupACKcount == 3$
- Langkah-langkah:
 - o $ssthresh = cwnd / 2$
 - o $cwnd = ssthresh + 3$
 - o Segmen yang hilang langsung dikirim ulang (retransmit)
 - o Selama menerima duplicate ACK, $cwnd$ terus meningkat
- Tujuan:
 - o Mempertahankan koneksi dan throughput tanpa kembali ke awal seperti pada timeout
- Keluar dari fast recovery:
 - o Saat menerima ACK baru $\rightarrow cwnd = ssthresh$ dan kembali ke Congestion Avoidance

- d. Apa peran "New ACK!" dalam mekanisme congestion control TCP seperti yang ditunjukkan pada gambar? Bagaimana TCP menggunakan informasi dari ACK untuk mengatur laju pengiriman data dalam berbagai fase?

New ACK menunjukkan bahwa data telah berhasil diterima oleh penerima.

- Fungsi utama:
 - o Memberi sinyal bahwa jaringan tidak sedang macet

TCP (TRANSMISSION CONTROL PROTOCOL)

- Digunakan untuk menyesuaikan nilai cwnd berdasarkan fase saat ini
- Pada setiap fase:
 - Slow Start: ACK baru → cwnd naik eksponensial
 - Congestion Avoidance: ACK baru → cwnd naik linier
 - Fast Recovery: ACK baru → menandakan akhir dari fase ini dan kembali ke congestion avoidance

B. TCP Performance Analysis

1. Seorang administrator jaringan ingin menganalisis performa TCP pada koneksi antara dua host (Client dan Server). Data diambil menggunakan Wireshark selama proses transfer file berukuran 10 MB melalui protokol TCP. Tujuannya adalah untuk:

a. Menghitung nilai RTT, EstimatedRTT, dan TimeoutInterval.

b. Mengestimasi TCP Throughput.

c. Menilai apakah koneksi memiliki performa yang optimal.

dengan data yang dikumpulkan setelah melakukan capture pada Wireshark adalah sebagai berikut:

No	Waktu Paket Dikirm (s)	Waktu ACK Diterima (s)	SampleRTT (ms)
1	0,000	0,050	50
2	0,051	0,102	51
3	0,103	0,155	52
4	0,156	0,209	53

Informasi tambahan:

- Ukuran file yang ditransfer: 10 MB
- Total waktu transfer: 2 detik.
- Window size TCP: 64 KB = 65,536 byte.
- Faktor bobot (α): 0.125 dan Faktor bobot (β): 0.25.

Jawaban dan Penjelasan :

a. Menghitung Nilai RTT, EstimatedRTT, dan timeoutInteval

- RTT (s)

1. $RTT = 0,050 - 0,000 = 0,050$

2. $RTT = 0,102 - 0,051 = 0,051$

3. $RTT = 0,155 - 0,103 = 0,052$

4. $RTT = 0,209 - 0,156 = 0,053$

- EstimatedRTT (ms)

1. $EstimatedRTT = (1 - 0,125) * 0 + 0,125 * 50 = 6,25$

2. $EstimatedRTT = (1 - 0,125) * 6,25 + 0,125 * 51 = 11,84375$

3. $EstimatedRTT = (1 - 0,125) * 11,84375 + 0,125 * 52 = 16,86328125$

4. $EstimatedRTT = (1 - 0,125) * 16,86328125 + 0,125 * 53 = 21,38037109375$

- DevRTT (ms)

TCP (TRANSMISSION CONTROL PROTOCOL)

1. $DevRTT = (1 - 0,25) * 0 + 0,25 * |50 - 6,25| = 10,9375$
2. $DevRTT = (1 - 0,25) * 10,9375 + 0,25 * |51 - 11,84375| = 17,9921875$
3. $DevRTT = (1 - 0,25) * 17,9921875 + 0,25 * |52 - 16,84375| = 22,283203125$
4. $DevRTT = (1 - 0,25) * 22,283203125 + 0,25 * |53 - 21,38037109375| = 24,6173095703125$

- TimeoutInterval (ms)

1. $TimeoutInterval = 6,25 + 4 * 10,9375 = 50$
2. $TimeoutInterval = 11,84375 + 4 * 17,9921875 = 83,8125$
3. $TimeoutInterval = 16,86328125 + 4 * 22,283203125 = 105,99609375$
4. $TimeoutInterval = 21,38037109375 + 4 * 24,6173095703125 = 119,849609375$

b. Mengestimasi TCP Throughput

- Average TCP Throughput (byte)

1. $Average\ TCP\ Throughput = 65,536 / 0,050 = 1.310.720\ \text{byte per detik}$
Konversi ke Mbps = $1.310.720 * 8 = 10.485.760\ (10,48576\ \text{Mbps})$
2. $Average\ TCP\ Throughput = 65,536 / 0,051 = 1.275.019,6$
Konversi ke Mbps = $1.275.019,6 * 8 = 10.280.156,86\ (10,28\ \text{Mbps})$
3. $Average\ TCP\ Throughput = 65,536 / 0,052 = 1.260.307,69$
Konversi ke Mbps = $1.260.307,69 * 8 = 10.082.461,54\ (10,08\ \text{Mbps})$
4. $Average\ TCP\ Throughput = 65,536 / 0,053 = 1.236.528,30$
Konversi ke Mbps = $1.236.528,30 * 8 = 9.892.226,42\ (9,89\ \text{Mbps})$

- TCP Throughput (byte)

1. $TCP\ Throughput = 1460 * 65,536 / 0,050 \sqrt{10^{-8}} = 19.136.512.000.000$
Konversi ke Mbps = $19.136.512.000.000 * 8 = 153.092.096.000.000\ \text{bps}\ (153,09\ \text{Tbps})$
2. $TCP\ Throughput = 1460 * 65,536 / 0,051 \sqrt{10^{-8}} = 18.761.286.274.509,80$
Konversi ke Mbps = $18.761.286.274.509,80 * 8 = 150.090.290.196.078,40\ (150,09\ \text{Tbps})$
3. $TCP\ Throughput = 1460 * 65,536 / 0,052 \sqrt{10^{-8}} = 18.400.492.307.692,31$
Konversi ke Mbps = $18.400.492.307.692,31 * 8 = 147.203.938.461.538,5\ (147,20\ \text{Tbps})$
4. $TCP\ Throughput = 1460 * 65,536 / 0,053 \sqrt{10^{-8}} = 18.053.311.320.754,72$
Konversi ke Mbps = $18.053.311.320.754,72 * 8 = 144.426.490.566.037,74\ (144,43\ \text{Tbps})$

c. Menilai apakah koneksi memiliki Performa yang optimal

Window Size = 65.536 byte:

RTT = 0,050 detik: 10,48 Mbps

TCP (TRANSMISSION CONTROL PROTOCOL)

RTT = 0,051 detik: 10,28 Mbps

RTT = 0,052 detik: 10,08 Mbps

RTT = 0,053 detik: 9,89 Mbps

Throughput rata-rata berkisar antara 9,89 Mbps hingga 10,48 Mbps, minimum 9,89 Mbps dan maksimum 10.48 Mbps, ini sudah cukup dan optimal untuk interaksi dengan internet

2. Saat ini seharusnya anda sudah paham bagaimana perhitungan berkaitan performansi TCP. Sekarang buka file TP.pcapng disana terdapat 12 paket. Lakukan hal perhitungan yang mirip dengan nomor 1 untuk menjawab pertanyaan berikut (pastikan time sudah dalam format Times of Days):

- Menghitung nilai RTT, EstimatedRTT, dan TimeoutInterval.
- Mengestimasi TCP Throughput.
- Menilai apakah koneksi memiliki performa yang optimal.

Jawaban dan Penjelasan:

A. Perhitungan RTT, EstimatedRTT, dan TimeoutInterval

RTT (s)

SampleRTT₁ = 0,020 detik

SampleRTT₂ = 0,030 detik

EstimatedRTT (s)

EstimatedRTT awal = SampleRTT₁ = 0,020 detik

EstimatedRTT = (1 - 0,125) * 0,020 + 0,125 * 0,030 = 0,875 * 0,020 + 0,125 * 0,030 = 0,0175 + 0,00375 = 0,02125 detik

DevRTT (s)

DevRTT awal = 0

DevRTT = (1 - 0,25) * 0 + 0,25 * |0,030 - 0,02125| = 0,75 * 0 + 0,25 * 0,00875 = 0 + 0,0021875 = 0,0021875 detik

TimeoutInterval (s)

TimeoutInterval = 0,02125 + 4 * 0,0021875 = 0,02125 + 0,00875 = 0,03 detik

B. Mengestimasi TCP Throughput

Average TCP Throughput (byte/s)

Window Size Rata-rata (16.384 bytes):

Average TCP Throughput = 16.384 / 0,02125 = 771.012,94 byte per detik

Konversi ke Mbps = 771.012,94 * 8 / 1.000.000 = 6,17 Mbps

Window Size Minimum (8.192 bytes):

Average TCP Throughput = 8.192 / 0,02125 = 385.506,47 byte per detik

Konversi ke Mbps = 385.506,47 * 8 / 1.000.000 = 3,08 Mbps

Window Size Maksimum (32.768 bytes):

Average TCP Throughput = 32.768 / 0,02125 = 1.542.025,88 byte per detik

Konversi ke Mbps = 1.542.025,88 * 8 / 1.000.000 = 12,34 Mbps

TCP Throughput dengan Probabilitas Kehilangan Paket (byte/s)

TCP (TRANSMISSION CONTROL PROTOCOL)

Window Size Rata-rata (16.384 bytes):

$$\text{TCP Throughput} = (1.460 * 16.384) / (0,02125 * \sqrt{0,001}) = 23.951.360 / (0,02125 * 0,0316) = 23.951.360 / 0,00067155 = 35.665.715,03 \text{ byte per detik}$$

$$\text{Konversi ke Mbps} = 35.665.715,03 * 8 / 1.000.000 = 285,33 \text{ Mbps}$$

Window Size Minimum (8.192 bytes):

$$\text{TCP Throughput} = (1.460 * 8.192) / (0,02125 * \sqrt{0,001}) = 11.960.320 / 0,00067155 = 17.809.696,64 \text{ byte per detik}$$

$$\text{Konversi ke Mbps} = 17.809.696,64 * 8 / 1.000.000 = 142,48 \text{ Mbps}$$

Window Size Maksimum (32.768 bytes):

$$\text{TCP Throughput} = (1.460 * 32.768) / (0,02125 * \sqrt{0,001}) = 47.841.280 / 0,00067155 = 71.240.691,91 \text{ byte per detik}$$

$$\text{Konversi ke Mbps} = 71.240.691,91 * 8 / 1.000.000 = 569,93 \text{ Mbps}$$

C. Menilai apakah koneksi memiliki Performa yang optimal

Perbandingan Throughput

Window Size rata-rata (16.384 bytes):

Average TCP Throughput: 6,17 Mbps

TCP Throughput dengan p: 285,33 Mbps

Window Size minimum (8.192 bytes):

Average TCP Throughput: 3,08 Mbps

TCP Throughput dengan p: 142,48 Mbps

Window Size maksimum (32.768 bytes):

Average TCP Throughput: 12,34 Mbps

TCP Throughput dengan p: 569,93 Mbps

Bandwidth-Delay Product (BDP)

$$\text{BDP} = 100 \text{ Mbps} * 0,02125 \text{ detik} = 2,125 \text{ Mbits} = 265.625 \text{ bytes}$$

Evaluasi Performa

Throughput meningkat signifikan (2-4 kali lipat) ketika window size ditingkatkan dari minimum ke maksimum, menunjukkan window size memiliki pengaruh besar pada performa koneksi.

Window size yang digunakan (8.192 - 32.768 bytes) jauh lebih kecil dari BDP optimal (265.625 bytes), mengindikasikan bahwa koneksi dibatasi oleh window size yang terlalu kecil.

RTT sebesar 21,25 ms termasuk dalam rentang normal untuk koneksi internet, namun kombinasi window size yang kecil menyebabkan throughput tidak optimal.

Meskipun koneksi dapat mentransfer data dengan throughput beberapa Mbps, koneksi belum mencapai performa optimal karena window size yang tidak memadai untuk memanfaatkan bandwidth yang tersedia.

Kesimpulan: Koneksi TCP yang dianalisis tidak memiliki performa yang optimal. Untuk meningkatkan performa, window size perlu ditingkatkan mendekati nilai BDP (sekitar 265 KB) untuk memaksimalkan throughput.

C. TCP Congestion Control

Jawab pertanyaan dibawah ini serta lakukan setiap langkah dibawah ini dan berikan screenshot-nya: Sebagai seorang administrator jaringan, Anda diminta untuk menganalisis komunikasi TCP antara Host A (172.16.2.55) sebagai client dan Server B (23.33.184.237) sebagai server menggunakan Wireshark. Data capture mencakup komunikasi TCP melalui port 80 (HTTP). Fokus analisis adalah memahami bagaimana mekanisme TCP Congestion Control bekerja dalam komunikasi ini, serta memvisualisasikan aliran data menggunakan Time-Sequence Graph

TCP (TRANSMISSION CONTROL PROTOCOL)

(Stevens) dari sudut pandang Client-to-Server dan Server-to-Client. Data capture didapatkan pada file TP2.pcapng

1. Identifikasi Parameter Awal TCP

- a. Temukan paket SYN pertama yang digunakan untuk memulai koneksi TCP!
Berapa nomor paketnya?
Berada pada paket 1

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.2.55	23.33.184.237	TCP	66	58155 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM

- b. Catat nilai MSS yang dikirim oleh Client dan Server (1460)

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.2.55	23.33.184.237	TCP	66	58155 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
2	0.054673	23.33.184.237	172.16.2.55	TCP	66	80 → 58155 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128

- c. Apakah ada perbedaan nilai MSS? Jika iya jelaskan kenapa bisa terjadi!
- Tidak ada perbedaan nilai MSS pada capture wireshark tersebut

2. Analisis Congestion Control

- a. Identifikasi apakah ada indikasi packet loss (misalnya, duplikasi ACK atau retransmisi) baik dari sisi client maupun server!

- Tidak ditemukan adanya retransmisi atau packet loss dalam file ini.

- b. Jika terjadi packet loss, bagaimana TCP merespons? Apakah Client mengurangi ukuran congestion window (cwnd)? Jelaskan proses recovery yang dilakukan

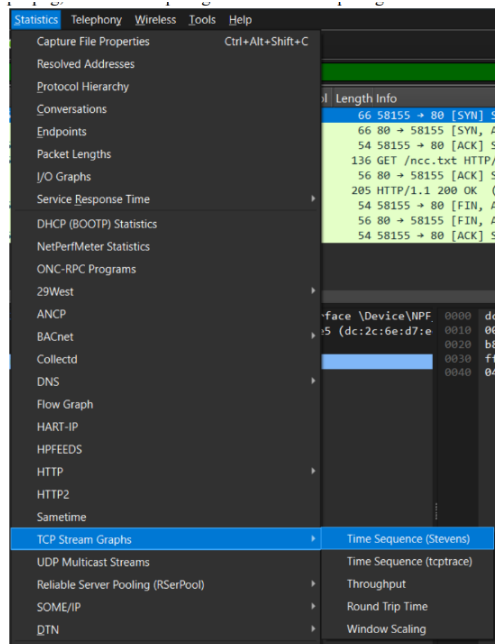
- Ketika terjadi packet loss pada TCP, TCP akan mengurangi ukuran congestion window (cwnd) untuk menghindari kemacetan jaringan dan memastikan pengiriman data yang efisien. Terdapat dua skenario utama saat terjadi packet loss pada TCP, yaitu:
 - Timeout: Ketika ACK tidak diterima dalam waktu yang ditentukan, TCP akan masuk ke fase slow start, dan cwnd akan dikurangi menjadi 1 MSS. Karena terjadinya timeout, TCP akan menurunkan throughput secara signifikan.
 - Fast Retransmit: Jika terjadi 3 ACK duplikat secara berturut-turut, TCP akan retransmit paket yang hilang tanpa menunggu timeout. Hal ini memungkinkan pengiriman paket yang hilang lebih cepat tanpa menunggu timeout terjadi.
 - Fast Recovery: Dalam kondisi ini, TCP tidak akan masuk ke fase slow start, namun cwnd akan dikurangi menjadi setengah dari ukuran sebelumnya dan TCP akan melakukan congestion avoidance dengan menaikkan cwnd secara perlahan untuk menghindari kemacetan.

- c. Temukan pola Slow Start, Congestion Avoidance, dan Fast Retransmit pada file tersebut (jika ada). Jika tidak ada, jelaskan kenapa hal tersebut tidak dapat terjadi!

- Tidak ditemukan pola slow start, congestion avoidance, dan fast retransmit pada file ini, karena setiap request yang dikirimkan mendapat respon ACK dalam waktu yang wajar, dan tidak ditemukan duplikasi pada request dan ACK tersebut. Hal ini menunjukkan bahwa tidak ada indikasi packet loss atau kemacetan yang cukup signifikan untuk memicu mekanisme tersebut.

TCP (TRANSMISSION CONTROL PROTOCOL)

3. Visualisasi Time-Sequence Graph (Stevens) Untuk memunculkan Time-Sequence Graph (Stevens) pada file TP2.pcapng, klik file TCP paling awal lalu ikuti seperti gambar berikut



Setelah melakukan gambar di atas akan muncul dialog box yang akan menjadi bahan untuk anda jawab pertanyaan berikut

- a. Pada dialog box memiliki fitur, jelaskan apa yang dimaksud dengan Stream dan Switch Direction

- Stream mengacu pada jumlah paket yang menggunakan IP address ini dan destination address, beserta informasi TCP yang sama. Karena pada file TP2 tidak ditemukan kejadian seperti retransmit atau duplicate ACK, maka stream pada paket pertama adalah 0.
- Switch Direction berarti mengganti arah dari IP address sumber ke IP address tujuan, dan sebaliknya. Ini juga bisa berarti menampilkan aliran data dari server ke client.

- b. Tampilkan Time-Sequence Graph (Stevens) untuk Client to Server, lalu amati pola aliran data pada grafik tersebut dan jawablah:

- Apakah ada indikasi Slow Start (peningkatan eksponensial Sequence Number)?
 - Terdapat 3 packet jump dengan ukuran yang sangat kecil. Ini menunjukkan adanya peningkatan eksponensial, yang bisa menjadi indikasi Slow Start.
- Apakah ada indikasi Congestion Avoidance (peningkatan linear Sequence Number)?
 - Tidak ditemukan indikasi Congestion Avoidance pada grafik tersebut, karena peningkatan sequence number tidak terjadi secara linear.
- Apakah ada anomali seperti retransmisi atau stagnasi Sequence Number?
 - Tidak ada anomali seperti retransmisi atau stagnasi pada sequence number yang terlihat pada grafik.

- c. Tampilkan Time-Sequence Graph (Stevens) untuk Server to Client, lalu amati pola aliran data pada grafik tersebut dan jawablah:

- Apakah ada indikasi Slow Start (peningkatan eksponensial Sequence Number)?

TCP (TRANSMISSION CONTROL PROTOCOL)

- Terdapat 1 packet jump dengan ukuran yang sangat kecil, namun hanya terjadi sekali. Ini bisa menjadi indikasi Slow Start.
- Apakah ada indikasi Congestion Avoidance (peningkatan linear Sequence Number)?
 - Tidak ditemukan indikasi Congestion Avoidance pada grafik ini.
- Apakah ada anomali seperti retransmisi atau stagnasi Sequence Number?
 - Tidak ditemukan adanya retransmisi atau stagnasi pada sequence number dalam grafik.

4. Kesimpulan

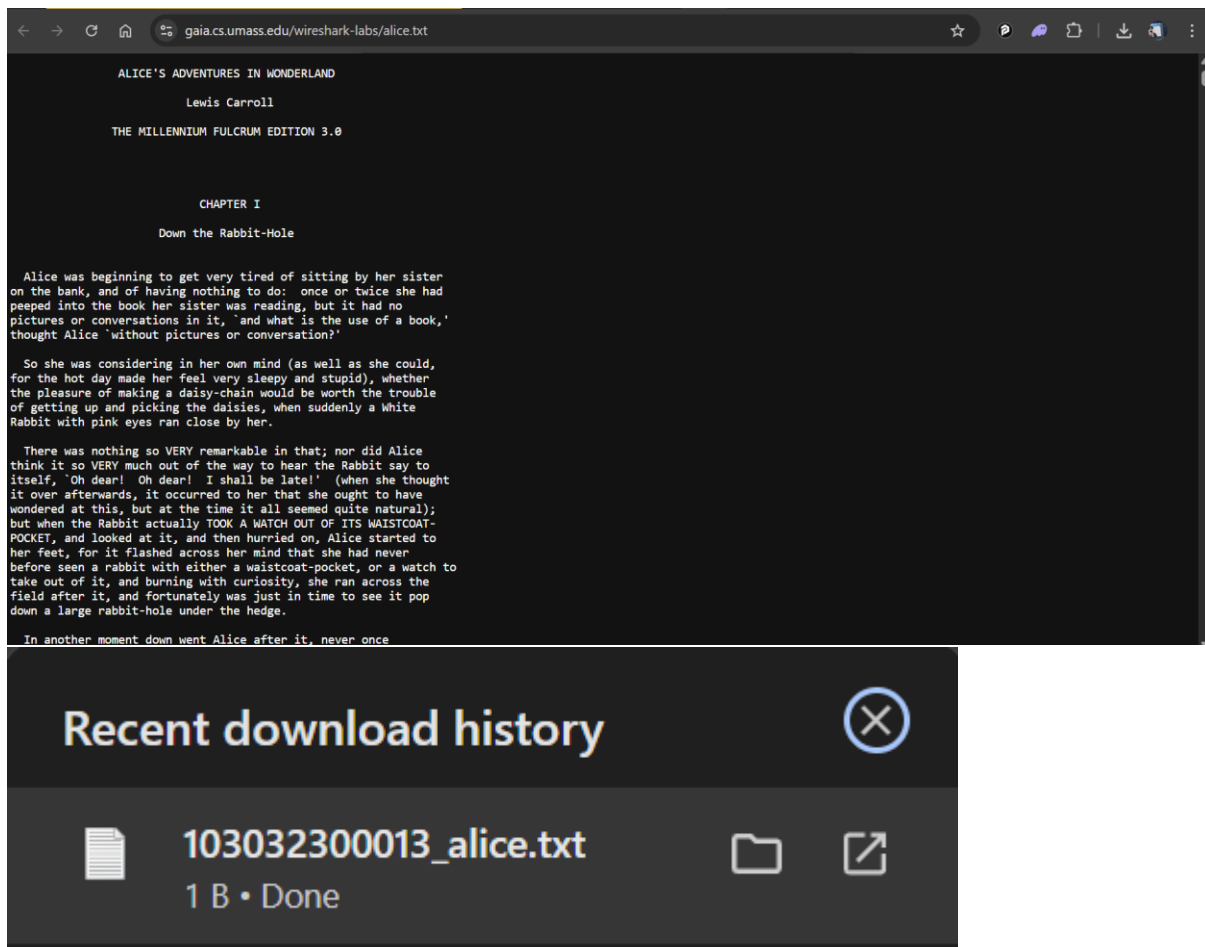
a. Berdasarkan analisis TCP Congestion Control dan visualisasi Time-Sequence Graph dari sudut pandang client-to-server dan server-to-client, apakah mekanisme TCP berjalan optimal dalam komunikasi ini?

- Ya, karena tidak ditemukan adanya duplicate ACK atau timeout ACK, dan waktu yang dibutuhkan untuk membangun koneksi TCP berlangsung dalam waktu yang relatif singkat.

b. Jika tidak, identifikasi masalah utama dan berikan rekomendasi untuk meningkatkan performa

- Seperti yang telah dijelaskan sebelumnya, komunikasi antara client dan server berjalan dengan respon yang cepat, tanpa adanya timeout atau duplicate ACK, yang berarti tidak terjadi packet loss.

D. Wireshark



JARINGAN KOMPUTER – TUGAS PENDAHULUAN MODUL 6

TCP (TRANSMISSION CONTROL PROTOCOL)



Upload page for TCP Wireshark Lab
Computer Networking: A Top Down Approach, 6th edition
Copyright 2012 J.F. Kurose and K.W. Ross, All Rights Reserved

If you have followed the instructions for the TCP Wireshark Lab, you have already downloaded an ASCII copy of Alice and Wonderland from <http://gaia.cs.umass.edu/wireshark-labs/alice.txt> and you also already have the Wireshark packet sniffer running and capturing packets on your computer.

Click on the Browse button below to select the directory/file name for the copy of alice.txt that is stored on your computer.

Choose File No file chosen

Once you have selected the file, click on the "Upload alice.txt file" button below. This will cause your browser to send a copy of alice.txt over an HTTP connection (using TCP) to the web server at gaia.cs.umass.edu. After clicking on the button, wait until a short message is displayed indicating the upload is complete. Then stop your Wireshark packet sniffer - you're ready to begin analyzing the TCP transfer of alice.txt from your computer to gaia.cs.umass.edu!!

Upload alice.txt file



Congratulations!

You've now transferred a copy of alice.txt from your computer to gaia.cs.umass.edu. You should now stop Wireshark packet capture. It's time to start analyzing the captured Wireshark packets!

137	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=7599 Win=1426 Len=0
138	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=79851 Win=1432 Len=0
139	4.913775	192.168.1.11	128.119.245.12	HTTP	24612	POST /wireshark-labs/lab3-1-reply.htm HTTP/1.1 (text/plain)
140	4.914013	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=80503 Win=1432 Len=0
141	4.914013	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=81055 Win=1432 Len=0

No.	Time	Source	Destination	Protocol	Length	Info
13	3.394621	35.186.224.45	192.168.1.11	TLSv1.2	94	Application Data
14	3.437667	192.168.1.11	35.186.224.45	TCP	54	62812 → 443 [ACK] Seq=44 Ack=41 Win=511 Len=0
15	3.628079	128.119.245.12	192.168.1.11	TCP	66	80 → 49472 [SYN, ACK] Seq=0 Ack=1 Win=29100 Len=0 MSS=1452 SACK_PERM WS=128
16	3.628954	192.168.1.11	128.119.245.12	TCP	54	49472 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
17	3.629581	128.119.245.12	192.168.1.11	TCP	54	80 → 49452 [ACK] Seq=1 Ack=2 Win=229 Len=0
20	4.151866	192.168.1.11	128.119.245.12	TCP	696	49467 → 80 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=642 [TCP PDU reassembled in 139]
21	4.152161	192.168.1.11	128.119.245.12	TCP	13122	49467 → 80 [ACK] Seq=643 Ack=1 Win=255 Len=13068 [TCP PDU reassembled in 139]
24	4.163085	192.168.1.11	74.125.200.136	QUIC	1292	Initial, DCID=5557e9314be2c5e8, PKN: 1, CRYPTO, CRYPTO, CRYPTO, PING, CRYPTO, CRYPTO
25	4.163212	192.168.1.11	74.125.200.136	QUIC	1292	Initial, DCID=5557e9314be2c5e8, PKN: 2, CRYPTO, PING, PING, PING, PADDING, PING, CRYPTO, PADDING...
26	4.163450	192.168.1.11	74.125.200.136	QUIC	117	0-RTT, DCID=5557e9314be2c5e8
27	4.170888	192.168.1.11	118.98.115.77	DNS	96	Standard query 0xe3fe A blowfish-blocklist-proxy.phantom.app
28	4.171049	192.168.1.11	118.98.115.77	DNS	96	Standard query response 0xe3fe A blowfish-blocklist-proxy.phantom.app
29	4.175785	118.98.115.77	192.168.1.11	DNS	128	Standard query response 0xe3fe A blowfish-blocklist-proxy.phantom.app A 104.18.35.227 A 172.64.1...
30	4.177335	118.98.115.77	192.168.1.11	DNS	169	Standard query response 0xe3fe A blowfish-blocklist-proxy.phantom.app HTTP5
31	4.178417	192.168.1.11	104.18.35.227	QUIC	1292	Initial, DCID=708c5837d112c18d, PKN: 1, CRYPTO, CRYPTO, CRYPTO, PING, PING, PING, PADDING, PING, ...
32	4.178513	192.168.1.11	104.18.35.227	QUIC	1292	Initial, DCID=708c5837d112c18d, PKN: 2, PING, PING, CRYPTO, CRYPTO, PING, PING, PING, PADDING, PING, ...
33	4.180321	74.125.200.136	192.168.1.11	QUIC	82	Initial, SCID=f557e9314be2c5e8, PKN: 1, ACK
34	4.181055	74.125.200.136	192.168.1.11	QUIC	1292	Initial, SCID=f557e9314be2c5e8, PKN: 2, ACK, PADDING
35	4.182283	74.125.200.136	192.168.1.11	QUIC	1292	Initial, SCID=f557e9314be2c5e8, PKN: 3, CRYPTO, PADDING
36	4.182283	74.125.200.136	192.168.1.11	QUIC	344	Protected Payload (KP0)
37	4.182283	74.125.200.136	192.168.1.11	QUIC	990	Protected Payload (KP0)
38	4.182283	74.125.200.136	192.168.1.11	QUIC	77	Protected Payload (KP0)
39	4.182683	192.168.1.11	74.125.200.136	QUIC	120	Handshake, DCID=f557e9314be2c5e8
40	4.182775	192.168.1.11	74.125.200.136	QUIC	73	Protected Payload (KP0), DCID=f557e9314be2c5e8
41	4.182947	192.168.1.11	74.125.200.136	QUIC	1288	Protected Payload (KP0), DCID=f557e9314be2c5e8
42	4.182981	192.168.1.11	74.125.200.136	QUIC	657	Protected Payload (KP0), DCID=f557e9314be2c5e8
43	4.195566	104.18.35.227	192.168.1.11	QUIC	1242	Initial, SCID=0156cad159d4ac2c14542fd15ed4942b091b0f25, PKN: 0, ACK
44	4.196386	104.18.35.227	192.168.1.11	QUIC	1242	Initial, SCID=0156cad159d4ac2c14542fd15ed4942b091b0f25, PKN: 1, ACK, CRYPTO
45	4.196386	104.18.35.227	192.168.1.11	QUIC	1242	Handshake, SCID=0156cad159d4ac2c14542fd15ed4942b091b0f25
46	4.196886	192.168.1.11	104.18.35.227	QUIC	1292	Protected Payload (KP0), DCID=0156cad159d4ac2c14542fd15ed4942b091b0f25
47	4.197111	192.168.1.11	104.18.35.227	QUIC	1292	Protected Payload (KP0), DCID=0156cad159d4ac2c14542fd15ed4942b091b0f25

Frame 139: 24612 bytes on wire (196896 bits), 24612 bytes captured (196896 bits) on interface
Ethernet II, Src: Intel_f0:7e:6d (9c:fc:e8:f0:7e:6d), Dst: zte_db:83:1c (f4:2d:06:db:83:1c)
Internet Protocol Version 4, Src: 192.168.1.11, Dst: 128.119.245.12

Frame (24612 bytes) Reassembled TCP (152976 bytes)

No.	Time	Source	Destination	Protocol	Length	Info
113	4.909217	192.168.1.11	128.119.245.12	TCP	2958	49467 → 80 [PSH, ACK] Seq=96475 Ack=1 Win=255 Len=2984 [TCP PDU reassembled in 139]
114	4.909564	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=44203 Win=922 Len=0
115	4.909564	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=45655 Win=945 Len=0
116	4.909564	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=47107 Win=967 Len=0
117	4.909564	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=48559 Win=990 Len=0
118	4.909585	192.168.1.11	128.119.245.12	TCP	11670	49467 → 80 [ACK] Seq=99379 Ack=1 Win=255 Len=11616 [TCP PDU reassembled in 139]
119	4.910128	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=50011 Win=1013 Len=0
120	4.910128	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=51463 Win=1036 Len=0
121	4.910152	192.168.1.11	128.119.245.12	TCP	5862	49467 → 80 [PSH, ACK] Seq=118995 Ack=1 Win=255 Len=5808 [TCP PDU reassembled in 139]
122	4.910568	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=52915 Win=1059 Len=0
123	4.910568	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=54367 Win=1081 Len=0
124	4.910568	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=55819 Win=1104 Len=0
125	4.910568	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=57271 Win=1127 Len=0
126	4.910583	192.168.1.11	128.119.245.12	TCP	11670	49467 → 80 [ACK] Seq=116803 Ack=1 Win=255 Len=11616 [TCP PDU reassembled in 139]
127	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=58723 Win=1150 Len=0
128	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=60175 Win=1173 Len=0
129	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=61627 Win=1196 Len=0
130	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=63079 Win=1218 Len=0
131	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=64531 Win=1241 Len=0
132	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=65983 Win=1264 Len=0
133	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=67435 Win=1287 Len=0
134	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=68887 Win=1310 Len=0
135	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=70339 Win=1332 Len=0
136	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=74695 Win=1400 Len=0
137	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=77599 Win=1426 Len=0
138	4.913666	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=79051 Win=1432 Len=0
139	4.913775	192.168.1.11	128.119.245.12	HTTP	24612	POST /wireshark-labs/lab3-1-reply.htm HTTP/1.1 (text/plain)
140	4.914013	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=80503 Win=1432 Len=0
141	4.914013	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=81055 Win=1432 Len=0
142	4.916181	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=83407 Win=1432 Len=0
143	4.916181	128.119.245.12	192.168.1.11	TCP	54	80 → 49467 [ACK] Seq=1 Ack=84859 Win=1432 Len=0

TCP (TRANSMISSION CONTROL PROTOCOL)

No.	Time	Source	Destination	Protocol	Length	Info
167	5.168414	128.119.245.12	192.168.1.11	TCP	56	80 → 49467 [ACK] Seq=140035 Win=1432 Len=0
168	5.168414	128.119.245.12	192.168.1.11	TCP	56	80 → 49467 [ACK] Seq=141487 Win=1484 Len=0
169	5.168669	128.119.245.12	192.168.1.11	TCP	56	80 → 49467 [ACK] Seq=142939 Win=1477 Len=0
170	5.168669	128.119.245.12	192.168.1.11	TCP	56	80 → 49467 [ACK] Seq=144391 Win=1500 Len=0
171	5.168942	128.119.245.12	192.168.1.11	TCP	56	80 → 49467 [ACK] Seq=145843 Win=1523 Len=0
172	5.169054	128.119.245.12	192.168.1.11	TCP	56	80 → 49467 [ACK] Seq=147295 Win=1546 Len=0
173	5.169315	128.119.245.12	192.168.1.11	TCP	56	80 → 49467 [ACK] Seq=148747 Win=1568 Len=0
174	5.169315	128.119.245.12	192.168.1.11	TCP	56	80 → 49467 [ACK] Seq=150199 Win=1591 Len=0
175	5.169684	128.119.245.12	192.168.1.11	TCP	56	80 → 49467 [ACK] Seq=151651 Win=1614 Len=0
176	5.169794	128.119.245.12	192.168.1.11	TCP	56	80 → 49467 [ACK] Seq=152977 Win=1637 Len=0
177	5.170048	128.119.245.12	192.168.1.11	HTTP	831	HTTP/1.1 200 OK (text/html)
178	5.217139	192.168.1.11	128.119.245.12	TCP	54	49467 → 80 [ACK] Seq=152977 Ack=778 Win=252 Len=0
179	7.488815	57.144.193.33	192.168.1.11	SSL	216	Continuation Data
180	7.493588	192.168.1.11	57.144.193.33	SSL	101	Continuation Data
181	7.511742	57.144.193.33	192.168.1.11	TCP	54	443 → 63009 [ACK] Seq=163 Ack=48 Win=2580 Len=0
182	9.428459	74.125.24.95	192.168.1.11	UDP	120	443 → 55021 Len=78
183	9.428459	74.125.24.95	192.168.1.11	UDP	64	443 → 55021 Len=22
184	9.436038	192.168.1.11	74.125.24.95	UDP	75	55021 → 443 Len=33
185	9.442262	192.168.1.11	118.98.115.77	DNS	91	Standard query 0xa727 A signaler-pa.clients6.google.com
186	9.442624	192.168.1.11	118.98.115.77	DNS	91	Standard query 0xa727 HTTPS signaler-pa.clients6.google.com
187	9.446142	118.98.115.77	192.168.1.11	DNS	107	Standard query response 0xa727 A signaler-pa.clients6.google.com A 64.233.170.95
188	9.446357	118.98.115.77	192.168.1.11	DNS	141	Standard query response 0x1142 HTTPS signaler-pa.clients6.google.com SOA ns1.google.com
189	9.447790	192.168.1.11	64.233.170.95	QUIC	1292	Initial, DCID=7aec5a7b07afcc72, PKN: 1, CRYPTO, CRYPTO, PING, CRYPTO
190	9.447888	192.168.1.11	64.233.170.95	QUIC	1292	Initial, DCID=7aec5a7b07afcc72, PKN: 2, CRYPTO, CRYPTO
191	9.448121	192.168.1.11	64.233.170.95	QUIC	120	0-RTT, DCID=7aec5a7b07afcc72
192	9.448608	192.168.1.11	64.233.170.95	QUIC	594	0-RTT, DCID=7aec5a7b07afcc72
193	9.466180	64.233.170.95	192.168.1.11	QUIC	82	Initial, SCID=fac5a7b07afcc72, PKN: 1, ACK
194	9.466180	64.233.170.95	192.168.1.11	QUIC	1292	Initial, SCID=fac5a7b07afcc72, PKN: 2, ACK, PADDING
195	9.467483	64.233.170.95	192.168.1.11	QUIC	1292	Initial, SCID=fac5a7b07afcc72, PKN: 3, CRYPTO, PADDING
196	9.467483	64.233.170.95	192.168.1.11	QUIC	344	Protected Payload (KP0)
197	9.467483	64.233.170.95	192.168.1.11	QUIC	990	Protected Payload (KP0)

1. Bagaimana cara mengidentifikasi paket TCP dalam hasil tangkapan Wireshark? Apa perbedaan antara paket TCP dan protokol lainnya (misalnya UDP) dalam hasil tangkapan? Tunjukkan baris pada Wireshark yang menunjukkan paket TCP berdasarkan aktivitas upload file Alice in Wonderland. Apakah ada indikasi bahwa koneksi TCP ditutup setelah proses upload selesai? Jelaskan.

- Paket TCP dapat diidentifikasi di Wireshark berdasarkan adanya request dan response dalam waktu yang singkat, dengan jumlah paket yang banyak. Protokol TCP biasanya juga menunjukkan adanya retransmisi dan duplicate ACK pada request TCP yang terlibat dalam proses tersebut, serta terdapat flag PSH (Push) dan FIN (Finish).
- Terdapat penutupan koneksi TCP setelah file berhasil di-upload dengan adanya RST, ACK pada respons TCP (terlihat di bagian bawah).
- Setelah pengiriman file ke server, terdapat situs HTTP POST, yang setelah file terkirim ke server, situs tersebut melakukan redirect yang menunjukkan bahwa file telah berhasil di-upload.

2. Jelaskan struktur header TCP dan identifikasi field penting yang berkaitan dengan aktivitas upload file. Bagaimana mekanisme kontrol kesalahan (error control) dalam TCP diimplementasikan melalui header TCP? Apakah ada indikasi penggunaan checksum dalam header TCP? Jelaskan kegunaannya.

- Struktur header TCP mencakup beberapa field penting, seperti Sequence Number, Acknowledgment Number, Flags, Window Size, Checksum, dan Urgent Pointer.
- Contohnya, terdapat duplicate ACK (DUP ACK) yang menunjukkan adanya packet loss. Ketika terjadi packet loss, TCP menggunakan fast retransmission dan melakukan recovery dalam waktu singkat untuk mengirimkan data yang hilang.
- Checksum pada header TCP digunakan untuk memverifikasi integritas data yang dikirimkan. Jika ada kesalahan selama transmisi, checksum membantu memastikan data yang diterima tidak rusak, dan jika ada duplikasi, TCP bisa memisahkannya, melakukan recovery, dan mengirim data yang benar.

3. Bagaimana data besar (file Alice in Wonderland) dikirim melalui TCP segment? Apakah data file dikirim dalam satu segment atau dibagi menjadi beberapa segment? Jelaskan. Tunjukkan contoh segment TCP dalam hasil tangkapan Wireshark dan jelaskan isi payload-nya. Bagaimana TCP segment menangani masalah seperti packet loss atau delay? Apakah ada indikasi adanya

TCP (TRANSMISSION CONTROL PROTOCOL)

retransmission dalam hasil tangkapan Anda? Jika ya, jelaskan penyebabnya. Apa yang terjadi jika salah satu segment gagal diterima oleh penerima? Bagaimana TCP memastikan integritas data?

- Data file dikirim dalam beberapa segment TCP untuk menghindari pengiriman data yang terlalu besar yang bisa menyebabkan delay. Setiap segment TCP biasanya memiliki ukuran sekitar 1466 bytes, namun ada juga segment yang lebih kecil, seperti 54 bytes.
- Terdapat retransmisi karena adanya packet loss, yang terjadi karena duplikasi data atau packet loss yang terdeteksi oleh TCP. Fast retransmission atau normal retransmission akan dipicu untuk mengirimkan ulang segment yang hilang dengan ukuran yang lebih kecil.
- Jika salah satu segment gagal diterima, TCP memastikan integritas data dengan retransmit data yang hilang, menghindari duplikasi dan memastikan semua data diterima dengan benar.

4. Jelaskan proses three-way handshake yang terjadi sebelum pengiriman data dimulai. Tunjukkan paket-paket yang membentuk three-way handshake dalam hasil tangkapan Wireshark. Apa peran flag SYN, SYN-ACK, dan ACK dalam proses ini? Jelaskan secara rinci. Bagaimana three-way handshake memastikan koneksi yang andal antara client dan server? Apakah ada indikasi bahwa koneksi TCP ditutup setelah proses upload selesai? Jelaskan bagaimana proses dilakukan.

- Proses three-way handshake dimulai dengan SYN yang dikirimkan oleh client ke server untuk meminta koneksi. Server merespons dengan SYN-ACK, yang berarti server menerima permintaan koneksi client dan mengonfirmasi bahwa koneksi dapat dilanjutkan. Setelah itu, client mengirimkan ACK sebagai tanda bahwa koneksi telah terbentuk.

Peran flag dalam three-way handshake:

- SYN (Synchronize): Digunakan untuk meminta koneksi ke server.
- SYN-ACK (Synchronize – Acknowledge): Menandakan konfirmasi server bahwa koneksi diterima dan siap untuk bertukar data.
- ACK (Acknowledge): Digunakan untuk menyelesaikan proses handshake dan memastikan bahwa kedua belah pihak siap untuk bertukar data.
- Three-way handshake memastikan koneksi yang andal antara client dan server dengan memverifikasi bahwa kedua belah pihak sepakat untuk memulai komunikasi dan dapat saling bertukar data.
- Setelah proses upload selesai, koneksi TCP ditutup menggunakan FIN (Finish) untuk mengakhiri komunikasi, dan RST (Reset) untuk mengatur ulang koneksi jika diperlukan. Setelah client mengirimkan FIN, server akan merespons dengan FIN-ACK, dan koneksi ditutup setelah itu.